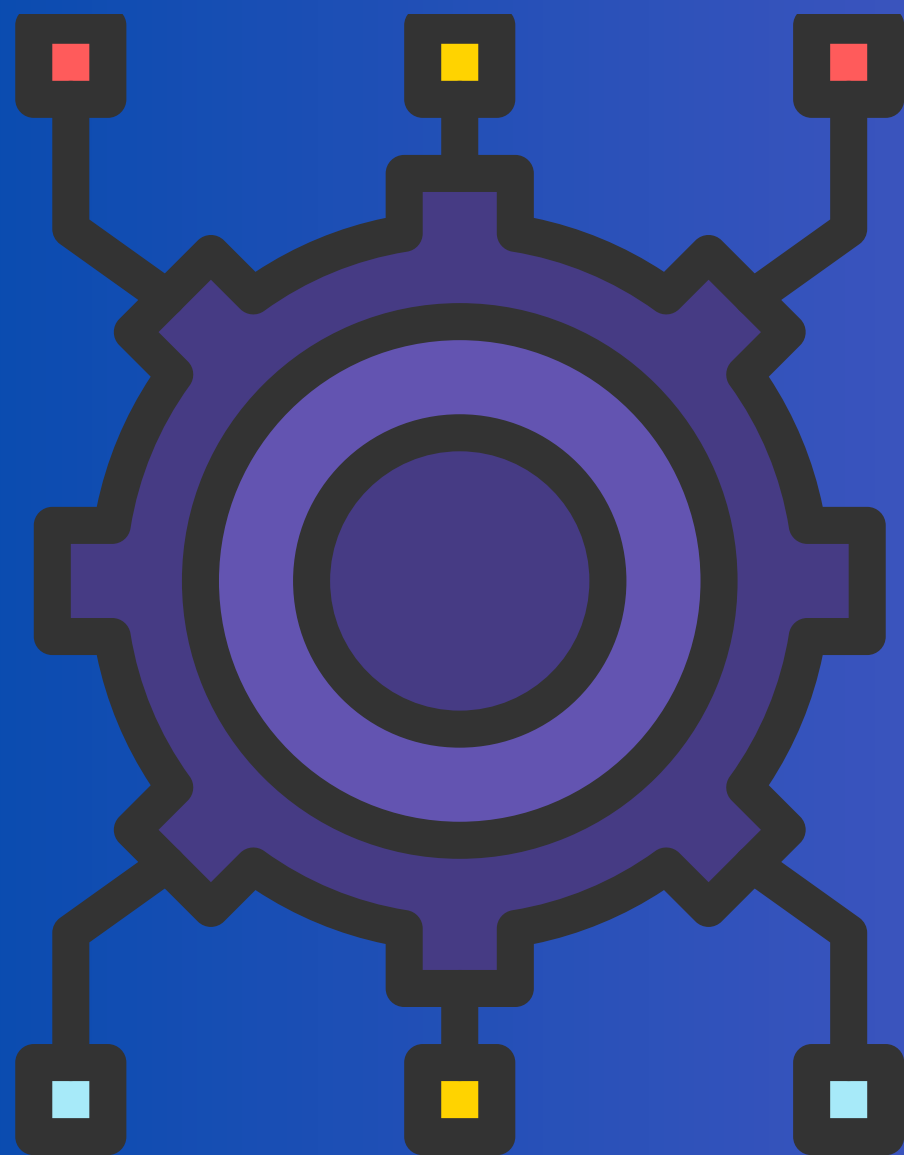




ALGORITHMS, DATA STRUCTURES, AND PROBLEM SOLVING

WHAT IS AN ALGORITHM?

WHAT IS AN ALGORITHM?

- AN **ORDERED** SET OF STEP-BY-STEP, **UNAMBIGUOUS**, AND **EXECUTABLE** INSTRUCTIONS THAT DEFINES A PROCESS TO SOLVE A SPECIFIC PROBLEM

AMBIGUOUS STATEMENT:

PLEASE CHECK THE AIR PRESSURE OF THE FRONT LEFT TIRE

UNAMBIGUOUS STATEMENT:

PLEASE CHECK THE AIR PRESSURE OF THE FRONT DRIVER'S SIDE TIRE

ALGORITHM REPRESENTATION

- REQUIRES WELL-DEFINED **PRIMITIVES**
- A COLLECTION OF PRIMITIVES CONSTITUTES A PROGRAMMING LANGUAGE

PRIMITIVES: THE SMALLEST STEPS A COMPUTER CAN DIRECTLY UNDERSTAND AND EXECUTE WITHOUT FURTHER BREAKDOWN

- EXAMPLES |

- ASSIGNING VALUES
- COMPARING VALUES
- DOING MATH OR LOGICAL OPERATIONS
- READING INPUT - GETTING VALUES FROM USER
- WRITING OUTPUT - PRINTING THE RESULT

SIMPLE RW EXAMPLE OF ALGORITHM

MAKING A CUP OF TEA

STEPS - HUMAN

1. BOIL WATER
2. PLACE TEA BAG IN CUP
3. POUR HOT WATER INTO CUP
4. STEEP THE TEA(WAIT FOR A FEW MINUTES)
5. REMOVE THE TEA BAG
6. ADD SUGAR OR MILK(IF DESIRED)
7. STIR THE TEA
8. SERVE AND ENJOY!

ALGORITHM - COMPUTER

1. **START**
2. BOIL WATER
3. PUT TEA BAG IN CUP
4. POUR BOILED WATER INTO CUP
5. WAIT FOR A FEW MINUTES
6. REMOVE TEA BAG
7. ADD SUGAR OR MILK(OPTIONAL)
8. STIR TEA
9. SERVE
10. **END**

EXAMPLE PRIMITIVES

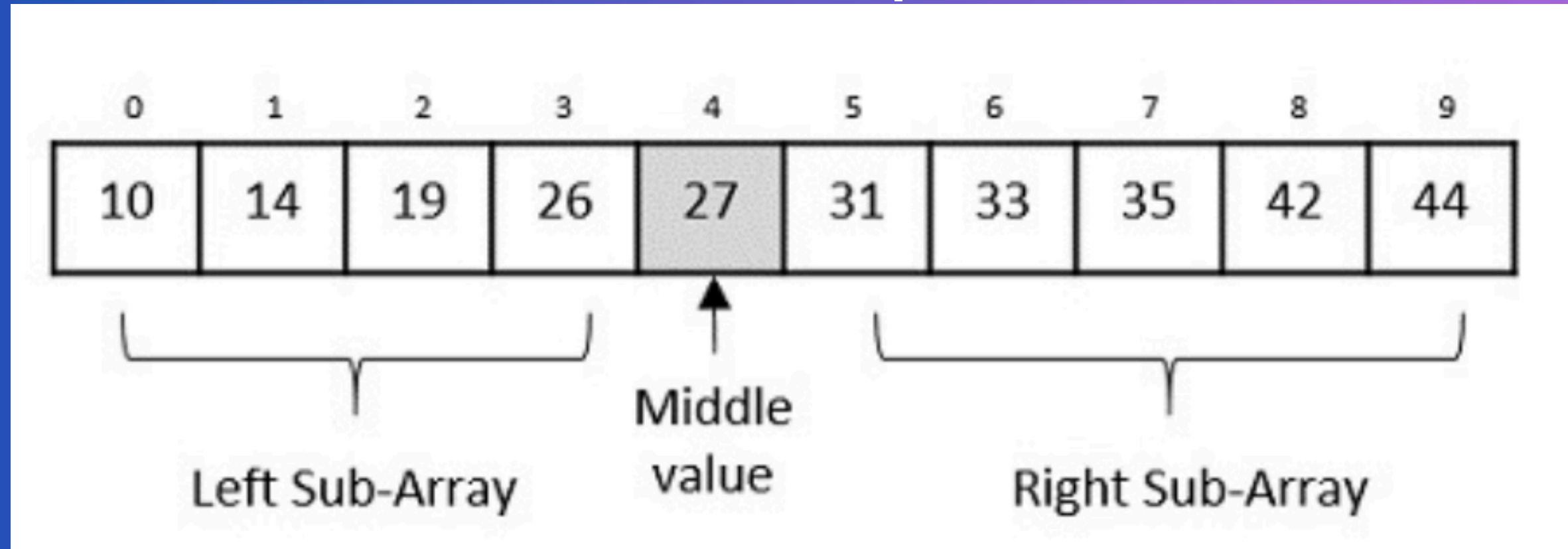
MAKING A CUP OF TEA PRIMITIVES

- 1.TURNING ON THE KETTLE (TO HEAT THE WATER)
- 2.PLACING TEA BAG IN CUP
- 3.MOVING WATER FROM KETTLE TO CUP
- 4.WAITING FOR A CERTAIN AMOUNT OF TIME
- 5.REMOVING TEA BAG FROM THE CUP
- 6.ADD INGREDIENT TO THE CUP
- 7.MIX INGREDIENTS USING A SPOON OR A STIRRER
- 8.HAND OVER CUP(TO DRINK TEA)

ALGORITHMS CATEGORIES

- **SEARCH**
 - SEARCH FOR AN ITEM IN A DATA STRUCTURE
- **SORT**
 - SORT ITEMS IN A DATA STRUCTURE
- **INSERT**
 - INSERT ITEM IN A DATA STRUCTURE
- **UPDATE**
 - UPDATE AN EXISTING ITEM IN A DATA STRUCTURE
- **DELETE**
 - DELETE AN EXISTING ITEM FROM A DATA STRUCTURE

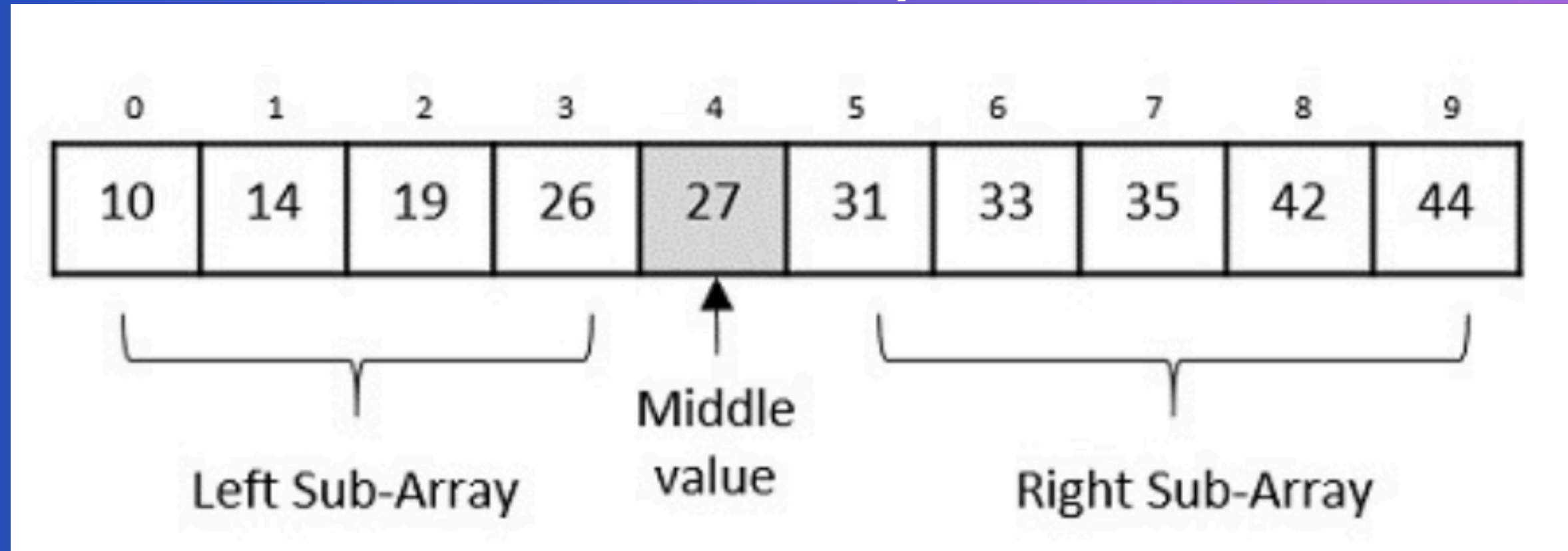
SEARCH ALGORITHM | BINARY SEARCH



- FINDING A TARGET VALUE IN A **SORTED** ARRAY BY REPEATEDLY DIVIDING THE SEARCH RANGE IN HALF, ELIMINATING HALF OF THE ELEMENTS IN EACH STEP UNTIL THE TARGET IS FOUND OR THE RANGE IS EMPTY.

WHY MUST THE ARRAY BE SORTED?

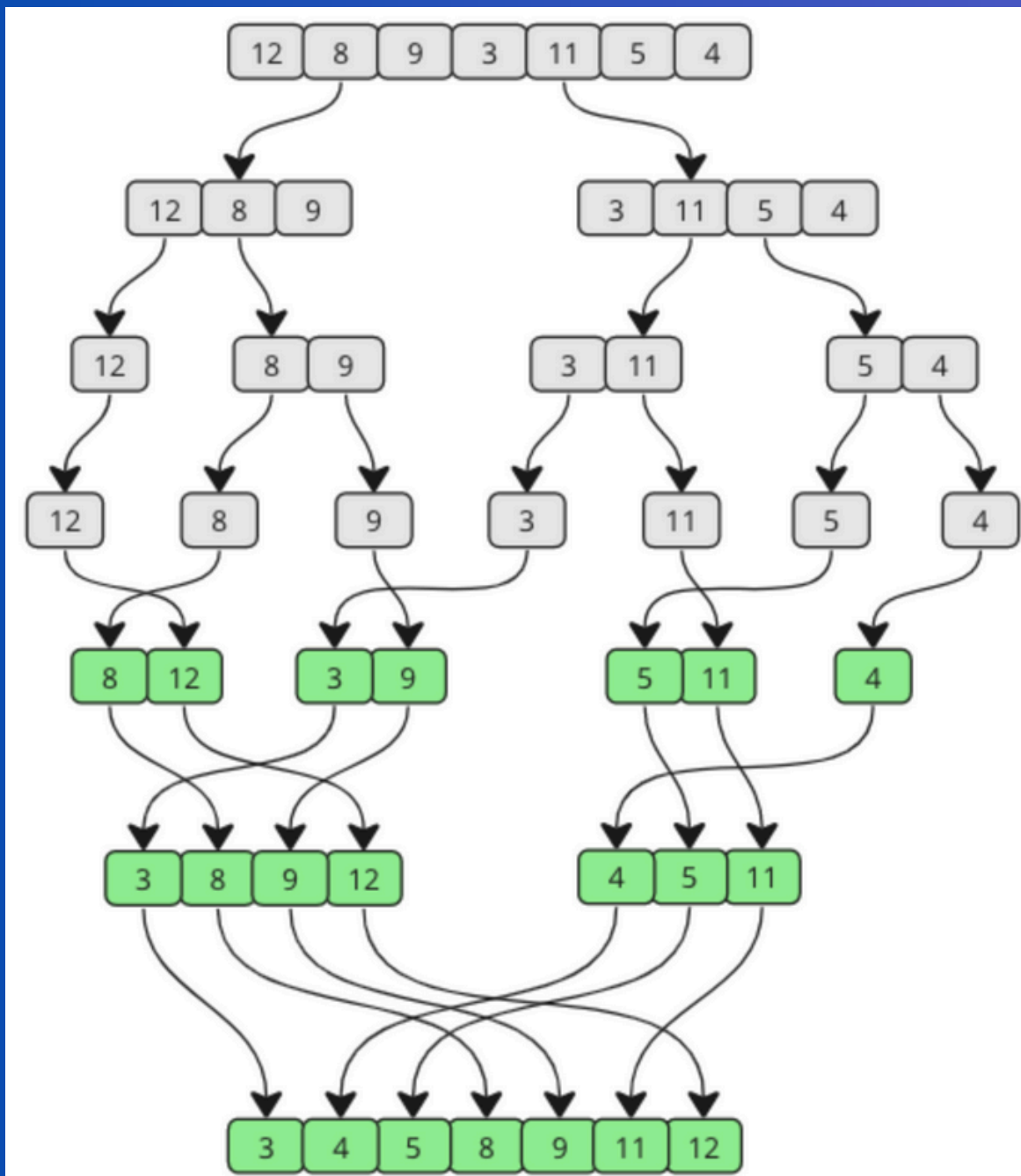
SEARCH ALGORITHM | BINARY SEARCH



- FINDING A TARGET VALUE IN A **SORTED** ARRAY BY REPEATEDLY DIVIDING THE SEARCH RANGE IN HALF, ELIMINATING HALF OF THE ELEMENTS IN EACH STEP UNTIL THE TARGET IS FOUND OR THE RANGE IS EMPTY.

BINARY SEARCH RELIES ON COMPARING THE TARGET WITH THE MIDDLE ELEMENT TO ELIMINATE HALF OF THE REMAINING ELEMENTS. IF THE ARRAY IS UNSORTED, THERE IS NO GUARANTEE THAT THE TARGET IS IN THE EXPECTED HALF, MAKING THE SEARCH UNRELIABLE.

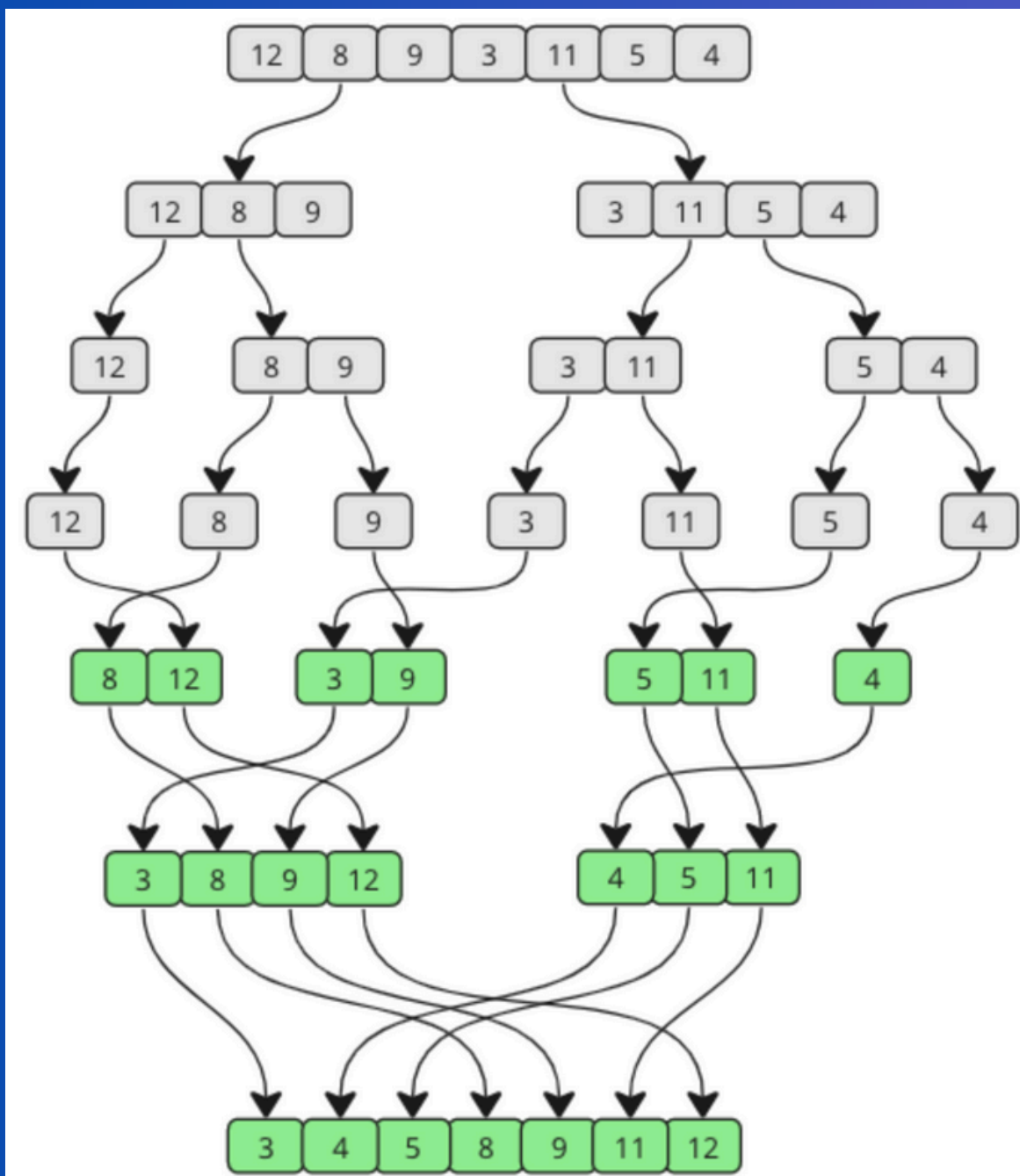
SORT ALGORITHM | MERGE SORT



A DIVIDE-AND-CONQUER SORTING ALGORITHM THAT SPLITS AN ARRAY INTO SMALLER SUBARRAYS, SORTS THEM RECURSIVELY, AND THEN MERGES THEM BACK TOGETHER IN ORDER.

WHAT IS RECURSION?

SORT ALGORITHM | MERGE SORT



A DIVIDE-AND-CONQUER SORTING ALGORITHM THAT SPLITS AN ARRAY INTO SMALLER SUBARRAYS, SORTS THEM RECURSIVELY, AND THEN MERGES THEM BACK TOGETHER IN ORDER.

A PROCESS WHERE A FUNCTION CALLS ITSELF TO SOLVE SMALLER INSTANCES OF A PROBLEM UNTIL A BASE CONDITION IS MET

WHAT IS PSEUDOCODE?

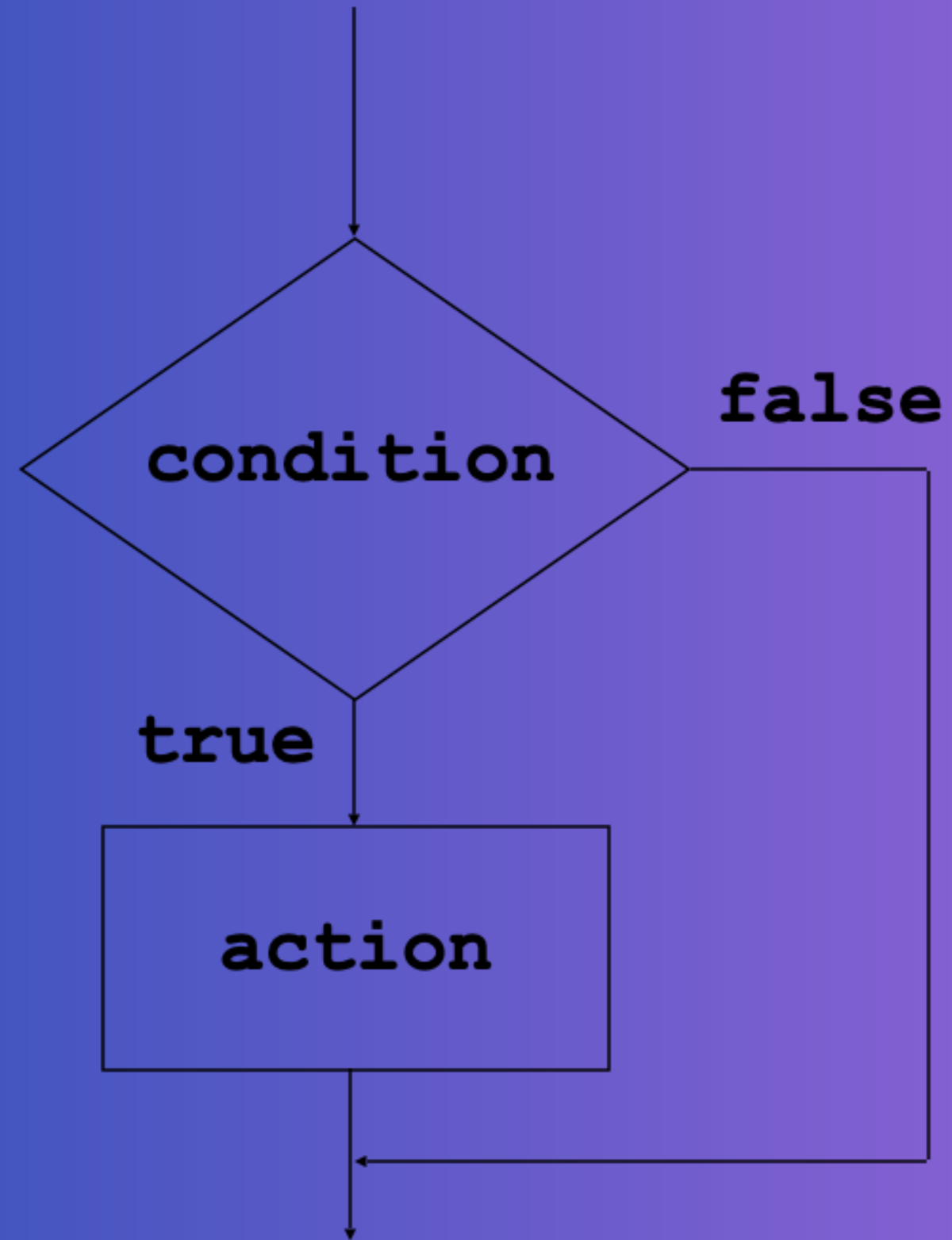
WHAT IS PSEUDOCODE?

- A NOTATIONAL SYSTEM FOR **INFORMALLY** EXPRESSING ALGORITHMS
- MUST BE CLEAR AND CONCISE
- PROGRAMMING LANGUAGE INDEPENDENT
 - A PROGRAMMER SHOULD BE ABLE TO USE ANY PROGRAMMING LANGUAGE TO IMPLEMENT THE PSEUDOCODE'S ALGORITHM

PSEUDOCODE PRIMITIVES

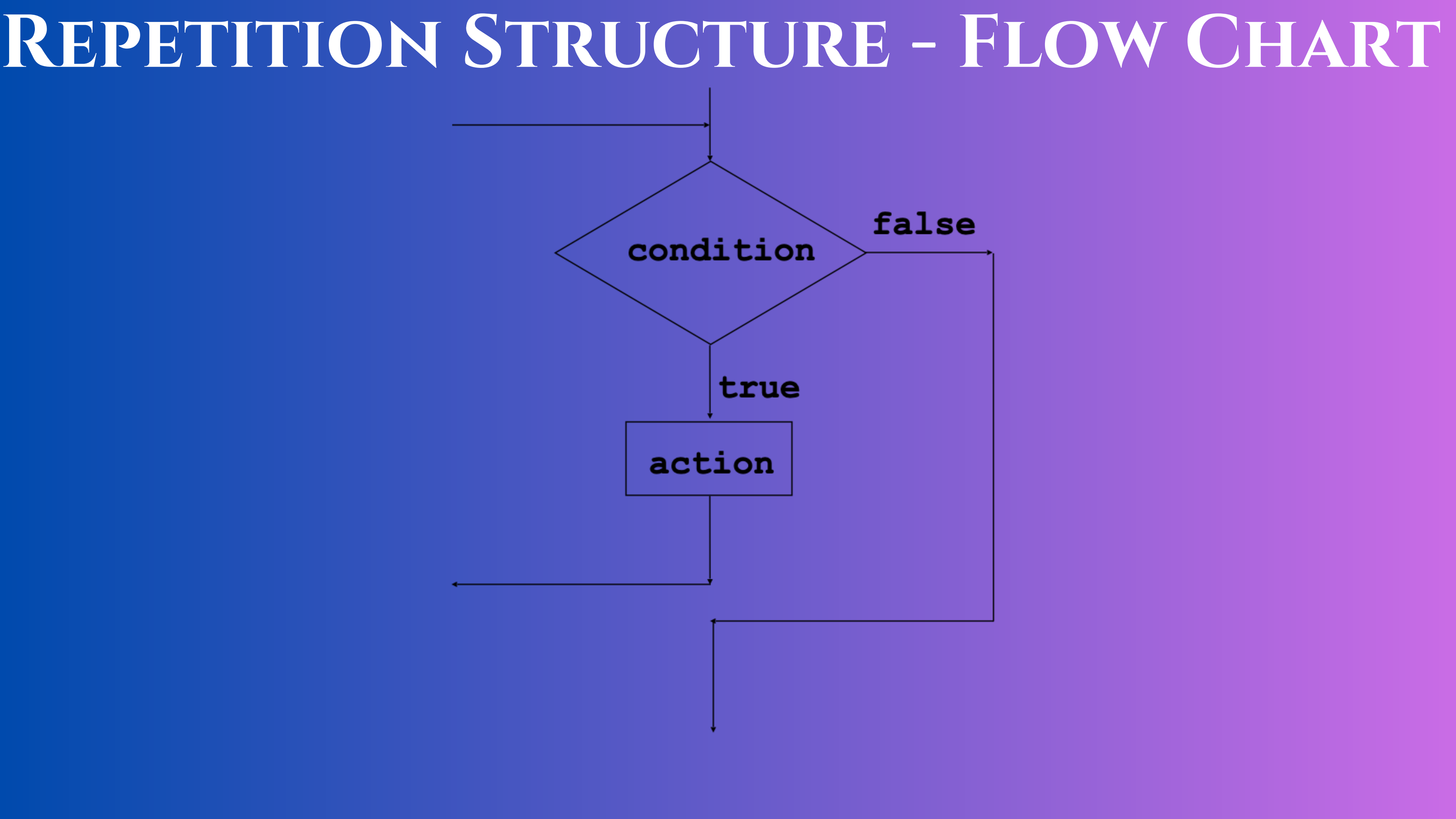
- ASSIGNMENT
 - **SET** NAME **TO** EXPRESSION
- CONDITIONAL SELECTION
 - **IF** CONDITION **THEN** ACTION
- - **IF** CONDITION **THEN**
 - ACTION 1
 - **ELSE**
 - ACTION 2

SELECTION STRUCTURE - FLOW CHART



PSEUDOCODE PRIMITIVES -CONT...

- REPEATED EXECUTION
 - **WHILE** CONDITION **DO** ACTIVITY
- PROCEDURE
 - **PROCEDURE** NAME (GENERIC NAMES)



COMPONENTS OF REPETITIVE CONTROL

INITIALIZE: ESTABLISH AN INITIAL STATE THAT WILL BE MODIFIED TOWARD THE TERMINATION CONDITION

TEST: COMPARE THE CURRENT STATE TO THE TERMINATION CONDITION AND TERMINATE THE REPETITION IF EQUAL

MODIFY: CHANGE THE STATE IN SUCH A WAY THAT IT MOVES TOWARD THE TERMINATION CONDITION

PSEUDOCODE EXAMPLES OF BINARY SEARCH

BINARYSEARCH(ARRAY, ELEMENT, START, FINISH)

IF START<=FINISH

MIDINDEX = (START + FINISH) / 2

IF ITEM == ARRAY[MIDINDEX]

RETURN MIDINDEX

ELSE IF ITEM < ARRAY[MIDINDEX]

RETURN BINARYSEARCH(AARRAY, ELEMENT, MIDINDEX + 1, FINISH)

ELSE

RETURN BINARYSEARCH(AARRAY, ELEMENT, START, MIDINDEX - 1)

RETURN -1

PSEUDOCODE EXAMPLE OF MERGE SORT

MERGESORT(ARRAY, LOW, HIGH)

IF LOW < HIGH

MID = (LOW + HIGH) / 2

MERGESORT(Array, LOW, MID)

MERGESORT(Array, MID + 1, HIGH)

MERGE(Array, LOW, MID, HIGH)

WHAT IS A DATA STRUCTURE?

WHAT IS A DATA STRUCTURE?

- A WAY OF ORGANIZING DATA SO DATA CAN BE USED EFFICIENTLY

PRIMITIVE

BASIC DATA STRUCTURES PROVIDED BY PROGRAMMING LANGUAGES TO REPRESENT SINGLE VALUES

SOME EXAMPLES

- INTEGERS
- FLOATS
- CHAR
- BOOLEAN

ABSTRACT

HIGHER-LEVEL DATA STRUCTURES THAT ARE BUILT USING PRIMITIVE DATA TYPES AND PROVIDE MORE COMPLEX AND SPECIALIZED OPERATIONS. SOME COMMON EXAMPLES OF ABSTRACT DATA STRUCTURES INCLUDE ARRAYS, LINKED LISTS, STACKS, QUEUES, TREES, AND GRAPHS.

SOME EXAMPLES

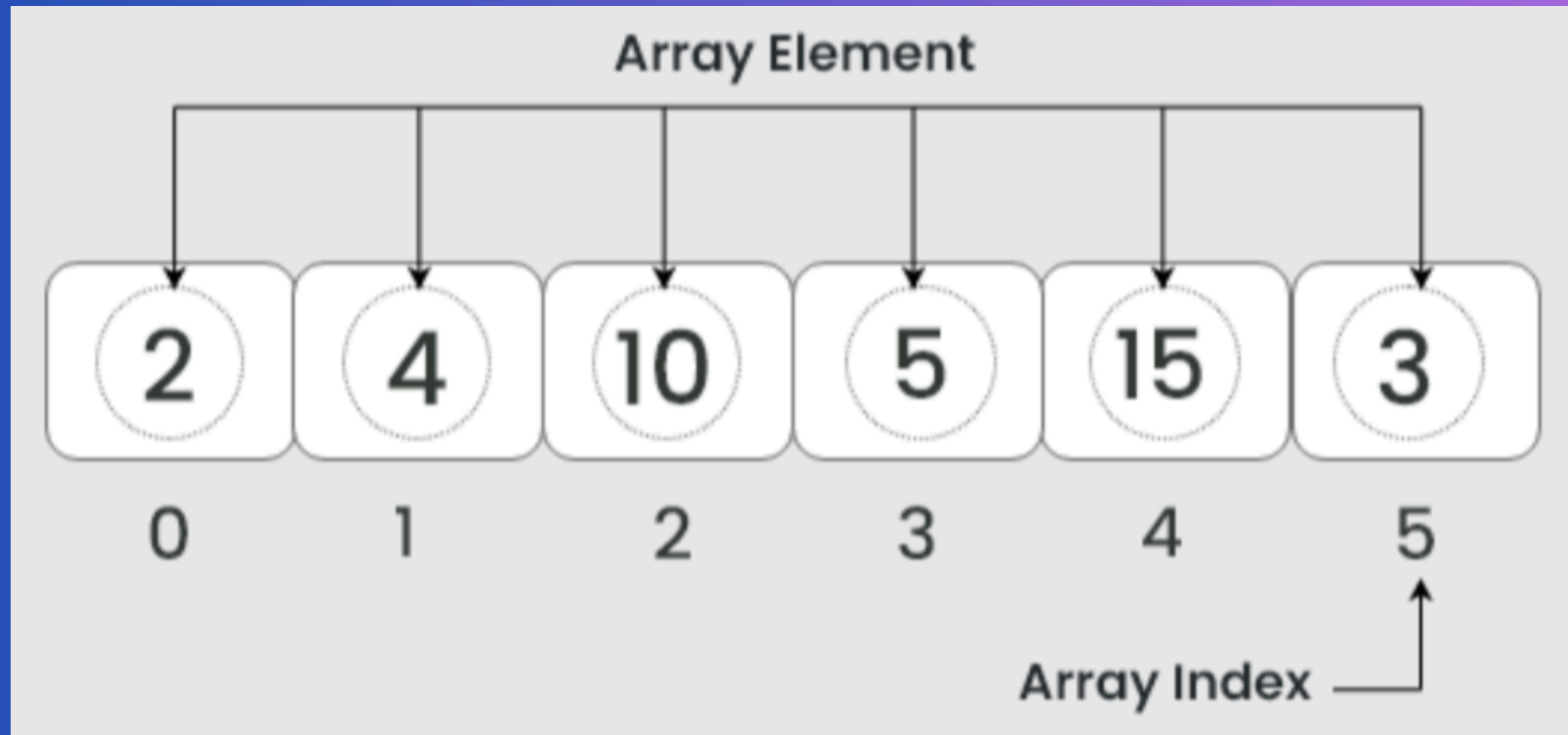
- ARRAY
- LINKED LIST
- STACK
- QUEUE

DATA STRUCTURES * ALGORITHMS

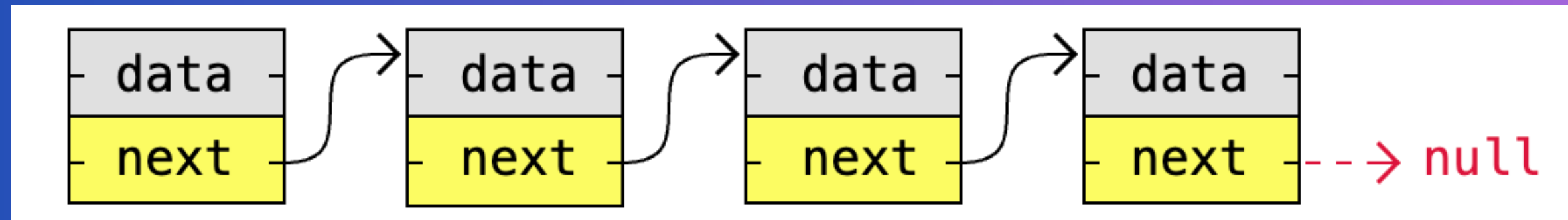
DATA STRUCTURES AND ALGORITHMS (DSA) **GO HAND IN HAND**. A DATA STRUCTURE IS NOT WORTH MUCH IF YOU CANNOT SEARCH THROUGH IT OR MANIPULATE IT EFFICIENTLY USING ALGORITHMS, AND THE ALGORITHMS ARE NOT WORTH MUCH WITHOUT A DATA STRUCTURE TO WORK ON.

DATA STRUCTURES 1 | ARRAY

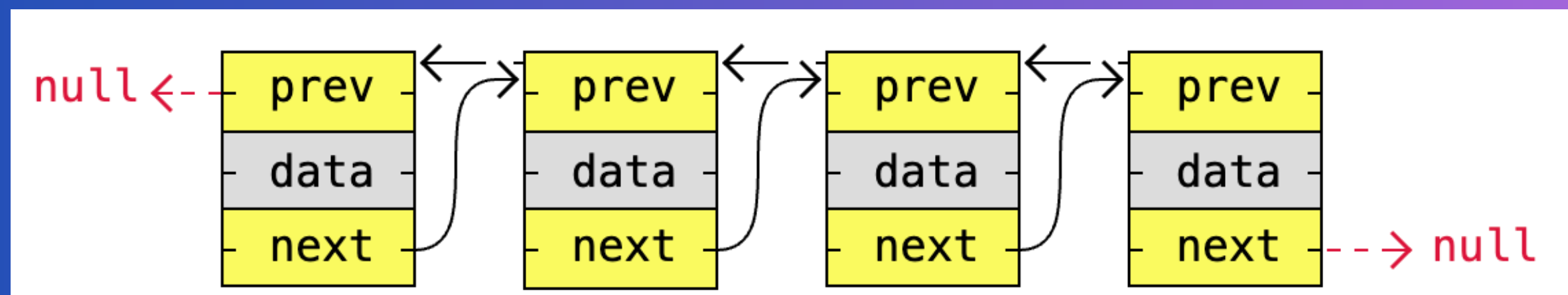
- ARRAYS ARE INDEXED, MEANING THAT EACH ELEMENT IN THE ARRAY HAS AN INDEX, A NUMBER THAT SAYS WHERE IN THE ARRAY THE ELEMENT IS LOCATED



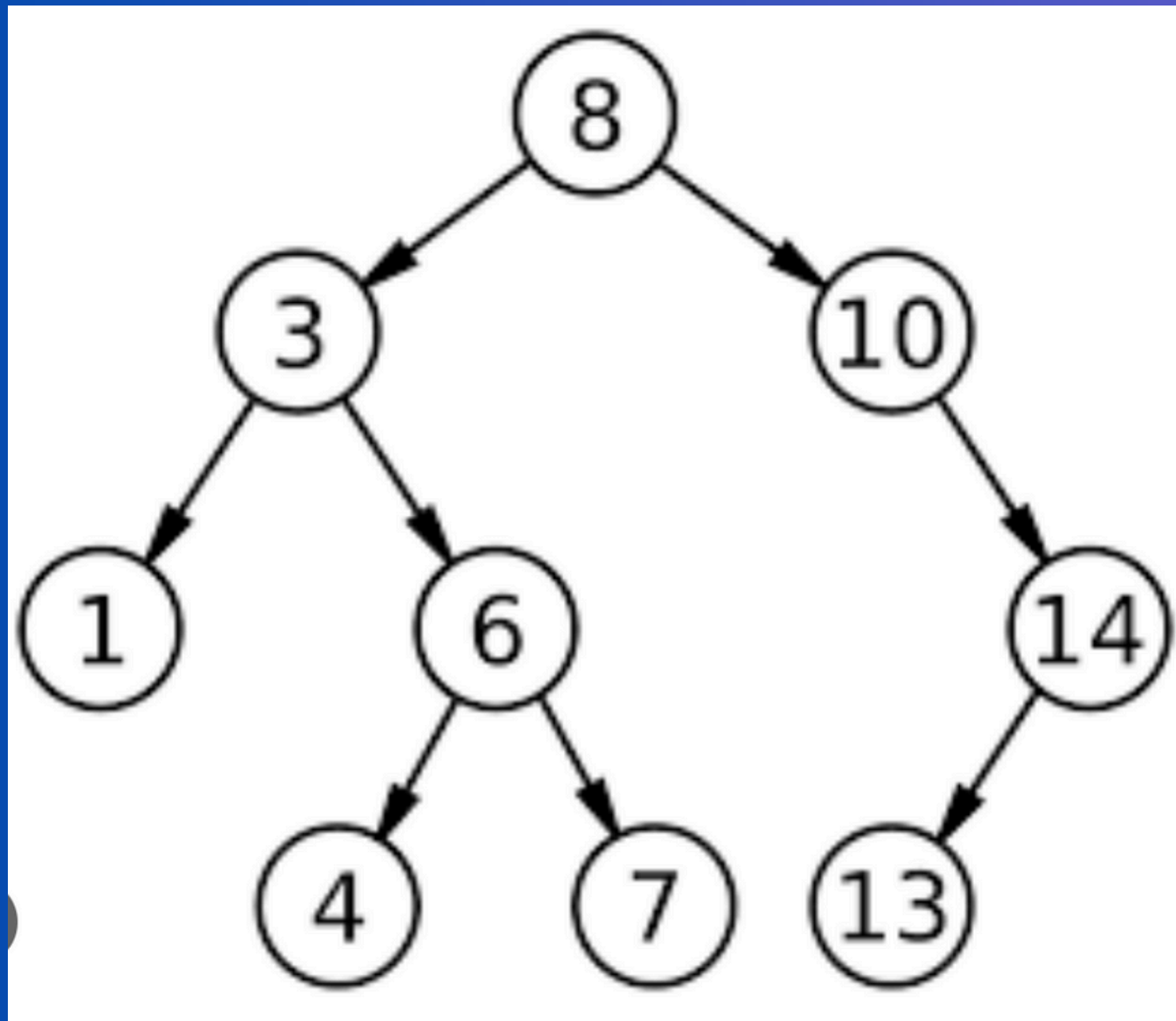
DATA STRUCTURES 2 | LINKED LIST



- A COLLECTION OF NODES WHERE EACH NODE CONTAINS DATA AND A REFERENCE (OR POINTER) TO THE NEXT NODE
- THREE TYPES OF LINKED LIST: SINGLY(ABOVE), DOUBLY(BELOW), CIRCULAR



DATA STRUCTURES 3 | BST



- ALLOWS US TO MAINTAIN A SORTED LIST OF NUMBERS
- A **BINARY** TREE BECAUSE EACH TREE NODE HAS A MAXIMUM OF TWO CHILDREN
- A **SEARCH** TREE BECAUSE IT CAN BE USED TO SEARCH FOR THE PRESENCE OF A NUMBER IN $O(\log(N))$ TIME

THE PROPERTIES THAT SEPARATE A BINARY SEARCH TREE FROM A REGULAR BINARY TREE IS

1. ALL NODES OF LEFT SUBTREE ARE LESS THAN THE ROOT NODE
2. ALL NODES OF RIGHT SUBTREE ARE MORE THAN THE ROOT NODE
3. BOTH SUBTREES OF EACH NODE ARE ALSO BSTs I.E. THEY HAVE THE ABOVE TWO PROPERTIES

WHAT IS A PROBLEM SOLVING?

WHAT IS PROBLEM SOLVING?

- THE PROCESS OF DESIGNING, ANALYZING, AND IMPLEMENTING EFFICIENT AND CORRECT SOLUTIONS TO COMPUTATIONAL PROBLEMS
- INVOLVES BREAKING DOWN COMPLEX TASKS INTO SMALLER, MANAGEABLE STEPS AND USING LOGICAL REASONING, ALGORITHMS, AND DATA STRUCTURES TO SOLVE THEM.

KEY ASPECTS

1. UNDERSTANDING THE PROBLEM
2. DESIGNING AN ALGORITHM
3. IMPLEMENTATION
4. ANALYSIS AND OPTIMIZATION
5. TESTING AND DEBUGGING
6. ITERATIVE IMPROVEMENT

PROBLEM SOLVING STRATEGIES

- **ASK QUESTIONS**

- WHAT DO I KNOW ABOUT THE PROBLEM?
- WHAT IS THE INFORMATION THAT I HAVE TO PROCESS IN ORDER TO FIND THE SOLUTION?
- WHAT DOES THE SOLUTION LOOK LIKE?
- WHAT SORT OF SPECIAL CASES EXIST?
- HOW WILL I RECOGNIZE THAT I HAVE FOUND THE SOLUTION?

- **REUSE SOLUTIONS. NEVER REINVENT THE WHEEL!**

- SIMILAR PROBLEMS COME UP AGAIN AND AGAIN IN DIFFERENT GUISES
- A GOOD PROBLEM SOLVER RECOGNIZES A TASK OR SUBTASK THAT HAS BEEN SOLVED BEFORE AND PLUGS IN THE SOLUTION

- **DIVIDE AND CONQUER!**

- BREAK UP A LARGE PROBLEM INTO SMALLER UNITS AND SOLVE EACH SMALLER PROBLEM
- APPLIES THE CONCEPT OF ABSTRACTION
- THE DIVIDE-AND-CONQUER APPROACH CAN BE APPLIED OVER AND OVER AGAIN UNTIL EACH SUBTASK IS MANAGEABLE

POLYA'S PROBLEM SOLVING STEPS - 1945

1. UNDERSTAND THE PROBLEM.
2. DEVISE A PLAN FOR SOLVING THE PROBLEM.
3. CARRY OUT THE PLAN.
4. EVALUATE THE SOLUTION FOR ACCURACY AND ITS POTENTIAL AS A TOOL FOR SOLVING OTHER PROBLEMS.